

重大 AI 更新提醒

07-22 | llama.cpp 发布 b10089，CUDA 端 GETROWS 支持 k-quant 量化嵌入查找

本期导读

本期重点：llama.cpp 发布 b10089，在 CUDA 后端为 GETROWS 操作添加了 k-quant 量化支持，显著提升设备端嵌入查找性能，避免回退到主机拷贝。

已检查来源

GitHub Release

1. ggml-org/llama.cpp release b10089: cuda: GETROWS quants (25962) cuda: add k-quant support to GETROWS

来源 GitHub Release | 类型 GitHub Release | 主题 检索增强与向量模型 时间 07-22 14:45 | 分数 7

是什么 这是 llama.cpp 项目的一个新版本发布（b10089），主要针对 CUDA 后端进行了优化，为 GETROWS 操作添加了 k-quant 量化支持。GETROWS 是用于设备端嵌入查找的关键操作，而 k-quant 是 GGUF 格式中常用的一种量化方法（如 Q4_K_M 将 token_embd 存储为 q6_K）。此前，当嵌入矩阵使用 k-quant 量化时，CUDA 后端会拒绝该操作，导致调度器回退到主机端，每次 token 生成时都需要将完整的嵌入矩阵拷贝回设备，严重影响性能。此次更新通过重构反量化器，在 CUDA 上实现了对 k-quant 的本地支持，避免了主机回退。

通俗解释 想象一下，你在玩一个大型语言模型，每次生成一个词时，都需要快速查找一个巨大的词嵌入表。如果这个表被压缩（量化）了，但你的 GPU 不直接支持这种压缩格式，它就只能把整个表从 GPU 内存拷贝到 CPU 内存，处理完再拷回来，这就像每次查字典都要把整本字典搬到另一个房间，非常慢。这次更新让 GPU 直接能处理这种压缩格式，省去了来回搬运，速度大幅提升。

主要作用 解决 CUDA 后端在设备端嵌入查找时因不支持 k-quant 量化而回退到主机拷贝的性能瓶颈，提升 llama.cpp 在 GPU 上的推理效率，尤其适用于使用量化模型的场景。

核心功能 为 GETROWS 操作添加了 k-quant（q2_K 到 q6_K）的 CUDA 支持，覆盖常见量化类型。；重构了反量化器，将 dequantize_block 内核中的超块反量化函数提取为共享设备函数，并复用于新的 k_get_rows_kq 内核。；每个线程块处理一个（目标行，超块）对，使用现有线程布局（q4_K 用 32 线程，其他 k-quant 用 64 线程）。；i-quant 支持作为待办事项，尚未实现。

对比判断 暂无可信公开对比数据，但根据提交信息，此前 CUDA 后端不支持 k-quant 的 GETROWS，导致回退到主机拷贝，每次 token 生成都要拷贝完整嵌入矩阵；更新后直接在设备端处理，预计显著降低延迟和带宽占用。

适合谁 适合使用 llama.cpp 在 NVIDIA GPU 上运行量化模型的开发者，尤其是使用 Q4_K_M 等 k-quant 量化方案的场景；对推理性能敏感的研究者和应用开发者。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b10089>

本简报由 AI 自动生成，仅供参考。